

?Design Discussions - Web Monitor Snapshot, Diff & Persistence Model

Purpose

Capture the design decision that emerged while comparing the monitoring policies described in the 2004 thesis with the behavior that actually survives in the Perl code, and translate that into a modern persistence model for web-monitor.

Historical observations

The thesis describes two caching policies. The archive policy stores timestamped snapshots at regular intervals whether or not anything changed; this preserves full history and supports arbitrary historical comparisons, but duplicates unchanged state. The overwrite policy stores a new copy only when content changes; this saves space, but the thesis explicitly notes that history is lost because the previous cached state is overwritten.

The recovered Perl strongly supports the researcher-domain archive path: timestamped snapshots can be written, reloaded, compared, and reported. Only low-level raw-cache helpers survive for the overwrite path; the course-domain comparison logic and conditional-overwrite orchestration are not present in Appendix A.

This matters for the modern port. Recreating only the fully recovered archive behavior would preserve what survived in code, but it would underuse an important design idea that was discussed in the thesis and only partially preserved in the implementation.

Decision for web-monitor

web-monitor will combine the useful properties of both historical policies.

Every scheduled crawl is recorded, but a new full snapshot is persisted only when the observed state differs from the most recent stored state.

A crawl with no change records that the site was checked at that time and still maps to the existing snapshot. A crawl that finds a change creates a new snapshot and records the transition from the previous state to the new one.

This avoids storing duplicate full snapshots on every crawl while still preserving historical state.

The practical effect is that a snapshot represents a distinct observed state, while crawl records establish when that state was known to be current.

Examples of questions this model supports

"State on September 16" resolves to the latest snapshot known to be in effect on September 16, supported by the crawl record for that date.

"What changed on September 16?" resolves to the change detected by that crawl, or "no change" if the crawl mapped to the existing state.

"What changed between September 16 and September 18?" resolves the state in effect at each date and compares those two states.

This is more direct than the historical overwrite policy, which loses prior state, and more storage-efficient than the archive policy, which stores duplicate state even when nothing changes.

Persisted model

Monitor

- URL and crawl configuration such as depth, allow/domain restrictions, schedule, and active status.

Crawl

- monitor
- crawled_at
- result/status: changed, no_change, or error
- snapshot representing the state observed during the crawl
- operational metadata such as pages crawled and failure details
- when changed, references/counts describing the transition from the prior snapshot

Snapshot

- monitor
- created_at / effective_from
- item count
- one record only for each distinct observed state

Snapshot Item

- snapshot
- URL
- title or description
- content hash and other normalized fields needed for comparison

A separate Change Event form is optional rather than required for the first implementation.

A changed Crawl can initially carry the prior/new snapshot references and added/removed/changed counts; this keeps the ActivityInfo model smaller while preserving the conceptual distinction between a crawl, a state, and a change.

ActivityInfo mapping

ActivityInfo is the persistent structured-data layer rather than the crawler or diff engine.

A minimal mapping is:

- Monitor as a top-level form
- Crawls as repeatable records associated with a monitor
- Snapshots as distinct state records associated with a monitor
- Snapshot Items as a subform/child collection of a snapshot

The Python application performs crawling, normalization, state comparison, and historical state resolution. ActivityInfo persists the resulting monitor configuration, crawl history, distinct snapshots, snapshot items, and summary counts. Its reporting layer can then summarize crawl frequency, change frequency, last crawl, last change, and related operational history.

Design consequence

The modern system is not a line-for-line Perl port. It preserves the historical monitoring ideas while making the storage model more explicit.

The incomplete recovery of the overwrite-policy implementation is useful context rather than a gap to hide: web-monitor brings that thesis-level design discussion back into the implementation and combines it with the fully recovered archive behavior in a way that supports both efficient persistence and historical queries.